

PairGraph: An Efficient Search-space-aware Accelerator for High-performance Concurrent Pairwise Queries

Yutao Fu[†], Zhongtian Long[†], Yu Zhang[†], Zirui He[†], Jin Zhao[†], Qiyuan Niu[†], Zixiao Wang[†], Hai Jin[†]

[†]National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Cluster and Grid Computing Lab, Huazhong University of Science and Technology, Wuhan, China

{fuyutao, longzt, zhyu, hezirui, zjin, qyniu, zwang62, hjin}@hust.edu.cn

Abstract—Pairwise queries have been widely used in many applications. Although several approaches have been recently proposed to accelerate a single query, they still suffer from irregular memory access and fragmented data sharing when processing *Concurrent Pairwise Queries* (CPQ) because of the poor temporal and spatial locality of traversal overlaps (i.e., graph structure data traversed by several queries). To address these challenges, this paper presents an accelerator named PairGraph to effectively support CPQ based on a novel *Search-space-aware Processing Model* (SPM). The key insight is the strong similarity of queries' search spaces, which are primarily concentrated on the graph topology between source and destination vertices. Consequently, our approach identifies the graph structure data traversed by most of the queries according to the graph topology between multiple pairs of vertices, and then fully reuses the data worth sharing to reduce off-chip communications. The experimental results indicate that PairGraph gains speedups of $5.59\times\sim14.25\times$ and $3.76\times\sim7.58\times$ compared with the state-of-the-art CPU-based system Gemini and the GPU-based system Gunrock, respectively. Compared with three cutting-edge accelerators, i.e., LCCG, ScalaGraph, and ReGraph, it gains speedups of $1.67\times\sim2.72\times$, $1.93\times\sim4.26\times$, and $2.66\times\sim4.28\times$, respectively.

I. INTRODUCTION

Pairwise queries are widely used in domains such as navigation systems [24], recommendation systems [25], and bioinformatics [22], where identifying relationships between two vertices on the graph is essential. Recently, several systems have been proposed to accelerate a single pairwise query [3], [4], [11]. However, as application demands grow, merely accelerating a single query is no longer sufficient to handle the high volume of concurrent query requests in practical scenarios. This has introduced the concept of *Concurrent Pairwise Queries* (CPQ), which aims to process multiple pairwise queries in highly concurrent environments.

Although many solutions [3]–[11] can support the CPQ execution, they still suffer from irregular memory access and fragmented data sharing for two reasons. First, CPQ exhibits traversal overlaps, as some portions of the graph are often traversed by multiple queries. However, these overlaps demonstrate low temporal locality because the same graph structure data is always accessed by queries at different times, leading to unnecessary memory accesses. Second, because of the complex topological structure between two vertices on the graph, each pairwise query holds a search space that covers an unknown subset of the graph. As a result, the traversal overlaps demonstrate low spatial locality. It means that the graph structure data traversed by most of the queries, namely *shared regions*, cannot be identified at runtime, leading to fragmented data sharing. Specifically, when CPQ is processed on general-purpose processors, the data within the shared regions is fragmented and scattered across different locations in the main memory, while all the traversed graph structure

data is loaded into the shared cache, including the data that can be only shared with a few queries. As a result, this data occupies the scarce on-chip memory but provides minimal sharing benefits.

Challenges come with opportunities. By analyzing the characteristics of the queries' search spaces, we have two key observations. First, although multiple queries access the graph structure data at different times, a significant portion of the data is identical, demonstrating *temporal similarity*. This indicates that many traversals of different queries can amortize the cost of accessing the same graph structure data if the data can be fully shared by all the queries. Second, most of queries' traversal paths are concentrated on the graph topology between the source and destination vertices, which shows *spatial similarity*. It indicates that the graph structure data traversed by most of the queries can be identified in the main memory according to the graph topology between multiple pairs of vertices before the queries are submitted.

Based on the above observations, in this paper, we propose PairGraph, a hardware accelerator based on a *Search-space-aware Processing Model* (SPM) to achieve efficient CPQ by fully exploiting the temporal and spatial locality of traversal overlaps. Unlike existing solutions, PairGraph employs a novel processing mechanism: shared region generation and overlap-driven processing. The former identifies the shared regions through tracking the graph topology bidirectionally between the source and destination vertices so as to leverage the spatial similarity of search spaces. In order to exploit the spatial locality of traversal overlaps, the overlap-driven processing mechanism leverages the data routing component to precisely fetch the data from the shared regions into the shared cache, while bypassing other non-shareable data, so as to coalesce the data worth sharing on-chip. Then the cached data is used to drive the CPQ execution to fully exploit temporal locality.

The experimental results show that PairGraph achieves speedups of $5.59\times\sim14.25\times$ and $3.76\times\sim7.58\times$ compared with the state-of-the-art CPU-based system Gemini [2] and the GPU-based system Gunrock [20], respectively. Compared with three cutting-edge accelerators, i.e., LCCG [5], ScalaGraph [12], and ReGraph [13], it gains speedups of $1.67\times\sim2.72\times$, $1.93\times\sim4.26\times$, and $2.66\times\sim4.28\times$, respectively.

II. BACKGROUND AND MOTIVATION

A. Concurrent Pairwise Queries

Pairwise Query Algorithms. Pairwise queries are lightweight graph processing algorithms that explore the relationship between two vertices on the graph. Unlike typical graph processing algorithms that traverse the entire graph, a pairwise query leverages the *pruning mechanism* to reduce the search space to a subset of the graph. Specifically, once a path connecting the source and destination vertices is identified, it serves to prune other paths. For instance, when querying the shortest path between two vertices, the length of a discovered path can be utilized to constrain subsequent query, as

Yu Zhang (zhyu@hust.edu.cn) is the corresponding author of this paper. This paper is supported by National Key Research and Development Program of China (No. 2024YFB4504200). This paper is also supported by NSFC (No. 62472183), Key Research and Development Program of Hubei Province (No. 2023BAB078). This work was supported by Ant Group through CCF-Ant Research Fund (No. CCF-AFSG RF20240204).

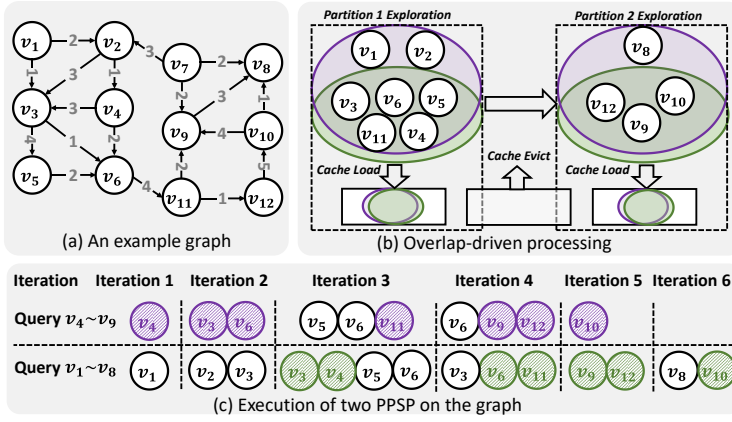


Fig. 1. An example of CPQ

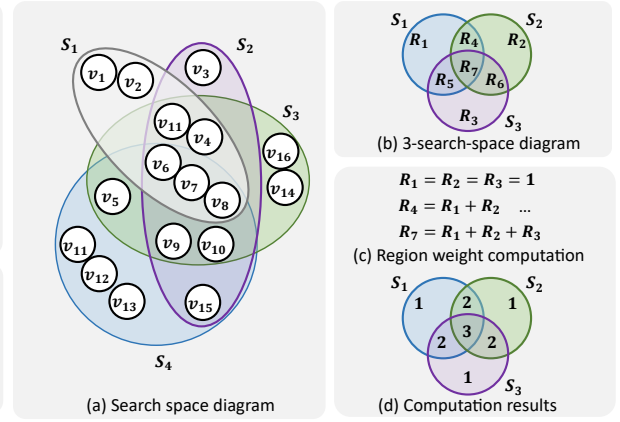


Fig. 2. An example of a search space diagram

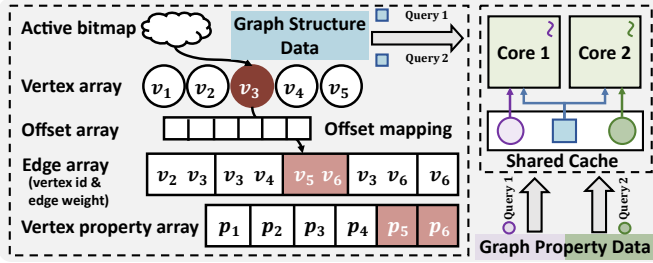


Fig. 3. Data access in CPQ under CSR storage structure

any longer paths can be disregarded. Consequently, pairwise queries can be defined as two-phase algorithms. The first phase aims to find a path from the source vertex to the destination vertex. Note that the path found in this phase is not necessarily optimal, and pruning does not occur during this phase. In the second phase, pruning begins once a path is found, enabling early termination of invalid traversals.

Concurrent Execution of Pairwise Queries. In practice, processing CPQ is more common than handling a single pairwise query because applications always require quick responses to handle a large number of queries in real-world data centers [5]. Concurrent execution has been shown to reduce inter-query blocking time [6], which enhances overall response speed. Consequently, this execution scheme is effective in improving the performance of pairwise query applications when facing thousands of query requests.

B. Limitations of Existing Solutions

There are two types of schemes that can support CPQ: one focuses on optimizing a single pairwise query, such as SGraph [4] and Pnp [3], while the other, including Glign [8] and LCCG [5], targets concurrent graph processing. However, neither type of scheme effectively supports CPQ, because the single-query scheme fails to consider data sharing between queries, while the concurrent-processing scheme overlooks the characteristics of pairwise queries. To investigate the causes of CPQ's high runtime overhead, we measure the performance of CPQ on Gemini, which is the state-of-the-art graph processing system, using different real-world graphs. Details of the baselines, datasets, and algorithms are presented in Section V. As shown in Fig. 4(a), we can observe that a significant portion of the execution time for processing CPQ is dedicated to memory access, while only a small portion is spent on computation. The results suggest that memory access is still the key bottleneck in processing CPQ, due to the following two problems.

Irregular Memory Access. Fig 1(c) depicts the accessing order of the vertices of two PPSP [4] queries on the illustrative graph in Fig. 1(a), where query $a \sim b$ denotes a query with source vertex a

and destination vertex b . We can observe that query $v_4 \sim v_9$ and query $v_1 \sim v_8$ traverse $v_3, v_4, v_6, v_9, v_{10}, v_{11}$, and v_{12} at different iterations. Consequently, as shown in Fig. 3, the vertex array, offset array, and edge array, which are collectively referred to as graph structure data in the widely used *Compress Sparse Row* (CSR) format [15], [16], are repeatedly accessed by CPQ, and the data is redundantly loaded into the shared cache, leading to significant off-chip communications. Note that the graph property data (i.e., the vertex property array in Fig.3) is private to each query and cannot be shared across queries. To quantify the impact, we conducted experiments to measure the redundant memory accesses caused by the irregular patterns during CPQ processing. As shown in Fig. 4(b), over 73.1% of memory accesses are found to be redundant.

Fragmented Data Sharing. Due to the diverse search spaces of pairwise queries, their traversal paths overlap in unknown portions of the graph. Consequently, the graph structure data accessed by most of the queries is fragmented within the main memory. This further leads to fragmented data sharing in CPQ and hinders the utilization of the shared cache. For example, vertices like v_1, v_2 , and v_8 , along with their edges in Fig. 1(a) should not be shared, as the search spaces of query $v_4 \sim v_9$ and query $v_1 \sim v_8$ do not overlap on these vertices. However, existing approaches still load these vertices and their edges into the shared cache, which may evict the data worth sharing. As shown in Fig. 4(c), we find that less than 13.9%, 1.1%, and 0.1% of the graph structure data can be shared by more than two, four, and eight queries during an iteration, respectively. This indicates that most of the data loaded into the shared cache is inefficiently shared. As a result, the cached data consumes the scarce on-chip memory but provides minimal benefits for data reuse.

C. Observations

We observe the search spaces of pairwise queries exhibit significant temporal and spatial similarity. It provides the opportunity to improve the performance of CPQ processing.

Temporal Similarity of Search Spaces. We observe that a large amount of the same graph structure data is accessed by various queries at different times. This suggests that if this data can be fully shared by all the queries in CPQ before being evicted from the shared cache, the off-chip communications can be significantly reduced. As shown in Fig 1(c), if the data traversed by query $v_4 \sim v_9$ (i.e., v_4, v_3 , and their edges) is pinned in the shared cache during iterations 1 and 2, then this data can be shared with query $v_1 \sim v_8$ during iteration 3. It motivates the use of data that needs to be shared to drive CPQ execution, thereby leveraging temporal similarity to reduce off-chip communications.

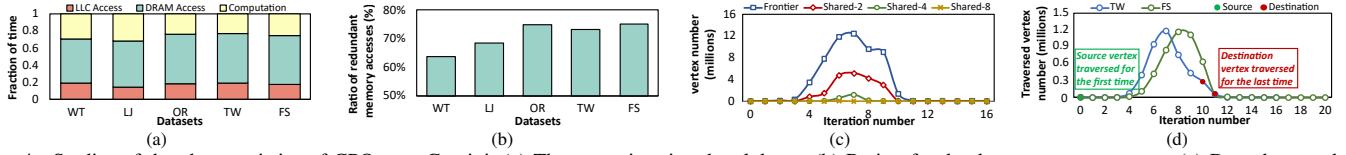


Fig. 4. Studies of the characteristics of CPQ over Gemini: (a) The execution time breakdown; (b) Ratio of redundant memory accesses; (c) Data that can be shared in each iteration on TW; (d) Spatial similarity of the search spaces for pairwise queries on TW and FS

Spatial Similarity of Search Spaces. We observe that the search spaces of pairwise queries exhibit notable spatial similarity. Fig. 4(d) illustrates that most traversals are made between the source and destination vertices, since any further exploration inferior to the existing optimal path is pruned after reaching the destination vertex. This indicates that, based on the position of the source and destination vertices in a pairwise query, a majority of the query’s traversal paths can be identified according to the graph topology between these two vertices. Consequently, the graph structure data traversed by most of the queries (i.e., shared regions) can be identified before the queries are submitted.

Based on the spatial similarity, shared regions on the graph can be modeled using the *search space diagram* as shown in Fig. 2(a). The search spaces of pairwise queries in CPQ, denoted as S_i , overlap on the graph and generate several regions. The importance of data across these regions is determined by the *region weight*, denoted as R_i in Fig. 2(b). Initially, the region weight for each query is set to 1, and the weights for new regions are computed using the operations as shown in Fig. 2(c). The computed result is [1, 1, 1, 2, 2, 2, 3] for $[R_1, R_2, R_3, R_4, R_5, R_6, R_7]$ in Fig. 2(d). Regions with higher region weight are more frequently accessed by queries, indicating that the data in these regions should be prioritized for sharing. The average region weight is used as the threshold for selecting the shared regions, which is demonstrated to be an effective way (see Section V-B).

III. SEARCH-SPACE-AWARE PROCESSING MODEL

Based on the observations above, we propose *Search-space-aware Processing Model* (SPM) to achieve efficient CPQ. It achieves temporal and spatial locality of traversal overlaps by generating shared regions on the graph and fully sharing the graph structure data within these regions.

A. Redundancy-free Shared Region Generation

The spatial similarity of search spaces motivates us to generate the shared regions based on the search space diagram. A useful method is bidirectionally tracking the graph topology between multiple pairs of source and destination vertices in a breadth-first manner. For each query, a sub-tracking propagates the region weight between the source and destination vertices. The new region weight is obtained by summing the propagated weights across vertices from several sub-tracking.

However, this straightforward scheme is inefficient due to redundant propagations. Assume that the region weight is propagated between v_4 and v_9 , and between v_1 and v_8 to generate the shared regions for query $v_4 \sim v_9$ and query $v_1 \sim v_8$ in Fig. 1(a). The propagations $v_1 \rightarrow v_2 \rightarrow v_4 \rightarrow v_3$ and $v_4 \rightarrow v_3$ are ongoing asynchronously, with the region weight being repeatedly propagated across the successors of v_4 . This leads to repeated access to the same edges and unnecessary computations on the same vertices. Luckily, there is an opportunity to gather the propagations at v_4 and deliver them to the following vertices all together in a single step. To address this, the process of generating shared regions is divided into two stages: the tagging stage and the propagating stage. In the tagging stage, the graph structure is traversed bidirectionally from multiple pairs of

source and destination vertices, with each edge being only traversed once. The number of times vertices are traversed, referred to as the tag, is recorded. Then in the propagating stage, the region weight is propagated and temporarily gathered at vertices with non-zero tags, as it indicates that some propagations have not yet arrived. Each time a propagation arrives at a vertex, the tag of the vertex decreases by one. The gathered propagation is forwarded only when the tag decreases to zero or when no other propagations are executing. This approach effectively eliminates redundant propagations, which enhances the efficiency of shared region generation.

Moreover, when generating shared regions, a path result between two vertices can be already obtained by propagating the path data along with the region weight. Although this result is not optimal, it can be used for pruning. This means that pruning can begin immediately when the queries are submitted, thereby reducing unnecessary computations.

B. Overlap-driven Processing

Based on the temporal similarity of search spaces, we adopt an overlap-driven processing approach to fully reuse the graph structure data within the shared regions. Specifically, the query process is coarse-grained into multiple steps of partition exploration. At each step, data from the shared regions is coalesced in the shared cache to drive the execution of multiple queries. The number of edges loaded from the shared regions is strictly controlled by the cache size to ensure that data from each exploration step can be contained within the cache. The cached data is pinned until all the queries have processed it, after which the execution proceeds to the next step.

The following one-step partition exploration illustrates the execution process described above. Assume that the cache can contain six edges. As shown in Fig. 1(b), for the query $v_4 \sim v_9$ and query $v_1 \sim v_8$, they first explore partition 1 in parallel, where the graph structure data in the shared regions (i.e., $v_4 \rightarrow v_3$, $v_4 \rightarrow v_6$, $v_3 \rightarrow v_5$, $v_3 \rightarrow v_6$, $v_5 \rightarrow v_6$, and $v_6 \rightarrow v_{11}$) are loaded into the cache and shared. Note that $v_1 \rightarrow v_2$, $v_1 \rightarrow v_3$, $v_2 \rightarrow v_3$, and $v_2 \rightarrow v_4$ are bypassed since they are identified as data outside the shared regions. After all the cached data is processed by the two queries, it is evicted and the execution proceeds to the next step of partition exploration.

C. Limitations of Implementing SPM on Typical Architectures

SPM faces significant challenges when implemented on general-purpose processors (i.e., CPU and GPU) since typical architectures present the following limitations. First, SPM relies on overlap-aware memory access mode, where data is cached or bypassed based on the region weight. However, such fine-grained control is not supported by typical architectures. Second, SPM requires pinning the data worth sharing on-chip, which is infeasible with the cache management strategies employed by typical architectures. Third, while SPM improves data sharing efficiency among queries at the algorithmic level, its performance is constrained by the prevalence of random memory accesses in CPQ. This issue, coupled with the low memory bandwidth utilization of typical architectures limits the overall performance gains. Consequently, it highlights the need for dedicated hardware architecture to fully realize the potential of SPM.

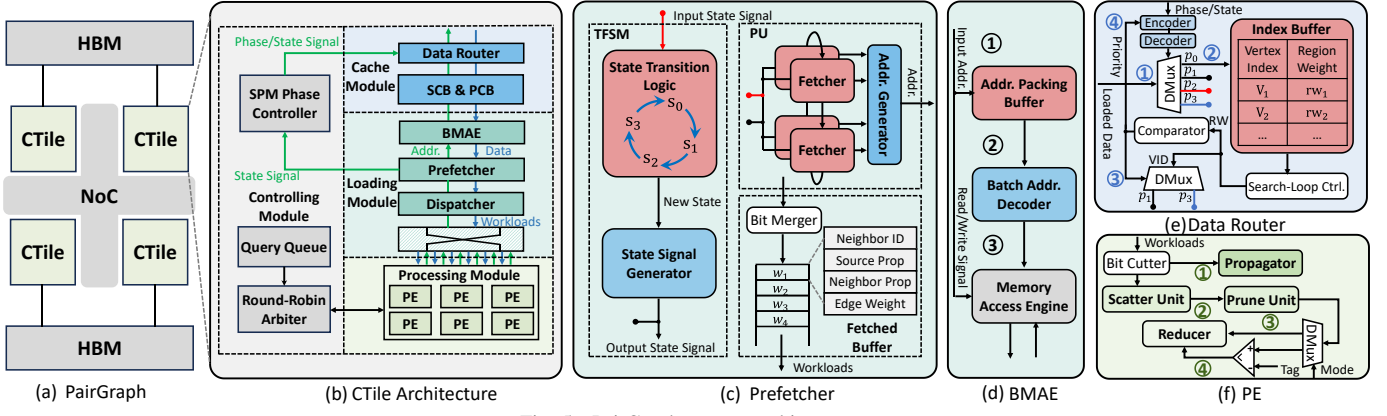


Fig. 5. PairGraph system architecture

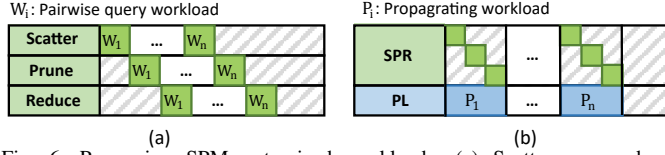


Fig. 6. Processing SPM-customized workloads: (a) Scatter-prune-reduce pipeline; (b) Propagating lane

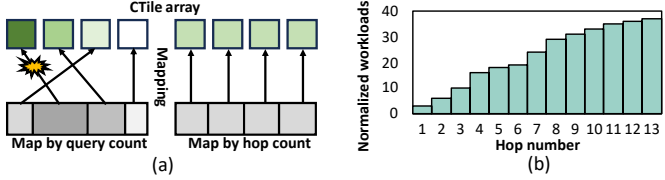


Fig. 7. Correlation-aware workload mapping: (a) Impact of different mapping strategies; (b) Correlation between workloads and hops

IV. PAIRGRAPH ARCHITECTURE

A. Architecture Overview

As shown in Fig. 5(a), PairGraph comprises the *Concurrent Tile Array*, *Network-On-Chip* (NoC), and *High-Bandwidth-Memory* (HBM). The system architecture of a *Concurrent Tile* (CTile) is shown in Fig. 5(b). It consists of the *Controlling Module*, *Processing Module*, *Loading Module*, and *Cache Module*.

PairGraph’s workflow aligns with the two phases of SPM, which first executes the *Redundancy-free Shared Region Generation* (RSG), and then maps the queries to CTiles for *Overlap-driven Processing* (OP). As shown in Fig. 5(b), the *Controlling Module* manages the execution phases by transmitting phase/state signals through the *SPM Phase Controller*, and schedules the queries in the *Query Queue* using the *Round-Robin Arbiter* to ensure execution fairness. The *Loading Module* achieves efficient memory access via the *Prefetcher* and *Batch Memory Access Engine* (BMAE). The *Prefetcher* leverages a task-level data prefetch pipeline to efficiently prefetch the graph data, while the BMAE applies a memory request packing approach to effectively utilize the memory bandwidth. The fetched workloads are evenly dispatched to the PEs in the *Processing Module* by the *Dispatcher*. And the PE is specifically designed to process the SPM-customized workloads. The *Cache Module* contains the *Shared Cache Block* (SCB) for graph-structure data sharing, and the *Private Cache Block* (PCB) for graph property data caching. To fully reuse the data worth sharing, the *Data Router* employs an overlap-aware data routing mechanism to route the data from the shared regions to the SCB and bypass other non-shareable data.

	1 bit	2 bit	1 bit	
	Exec Phase	Pref State	Priority	
code 1	RSG (0)	\	\	SCB
code 2	OP (1)	S_0 (00)	\	INB
code 3	OP (1)	S_1 (01)	1	SCB
code 4	OP (1)	S_2 (10)	1	SCB
code 5	OP (1)	S_3 (11)	\	PCB
code 6	OP (1)	S_1 (01)	0	Bypass
code 7	OP (1)	S_2 (10)	0	Bypass

Fig. 8. Routing code

B. Graph Data Prefetching and Memory Request Packing

Graph Data Prefetching. We developed a task-level prefetch pipeline consisting of four sub-tasks to efficiently fetch graph data. As shown in Fig. 5(c), the *Traversal Finite State Machine* (TFSM) manages the four states (i.e., S_0 : fetch_source, S_1 : fetch_offsets, S_2 : fetch_edges, and S_3 : fetch_properties) of the *Prefetch Unit* (PU) by generating state signals. Under TFSM control, the PU first gets the active vertex index in S_0 , followed by fetching edge offsets in S_1 . In S_2 , it iteratively prefetches 16 B of the associated edge data based on the offsets. Finally, in S_3 , it fetches the vertex property data of the current active vertex and its neighboring vertices. The fetched data is merged into a workload and transmitted to the *Dispatcher* via the *Fetched Buffer*.

Memory Request Packing. When processing CPQ, off-chip memory traffic generated during an iteration in SPM can be substantial, due to the *power-law property* [5] and concurrent execution. With 36 concurrent pairwise queries, we find that the traffic can reach 280 KB~3.9 MB per iteration. However, this traffic is always fragmented into very small memory requests, each request only asking for a few bytes of data. This leads to extremely low memory bandwidth utilization in HBM (about 2%~4%), thereby constraining the overall performance. To better utilize the memory bandwidth, the BMAE packs the memory requests during an iteration. Specifically, it merges multiple memory requests to generate a single request asking for 96 KB of data. As shown in Fig. 5(d), the input addresses are temporarily stored in the *Address Packing Buffer* and counted (①). Once the count reaches the limit, addresses are decoded in a batch (②) and sent to the *Memory Access Engine* (③). The packed addresses are then put on the address bus, and all the corresponding data is accessed in one go. Note that when there are no addresses streaming into the *Address Packing Buffer*, the remaining addresses are directly issued.

C. Overlap-aware Data Routing

To exploit the spatial locality of the traversal overlaps, we designed the *Data Router* with an overlap-aware data routing mechanism. In the RSG phase, data is directly cached in SCB, because all the data is useful for generating the shared regions. In the OP phase, when the queries are submitted, only the graph structure data in the shared regions is cached.

The workflow of the *Data Router* is shown in Fig. 5(e). Once

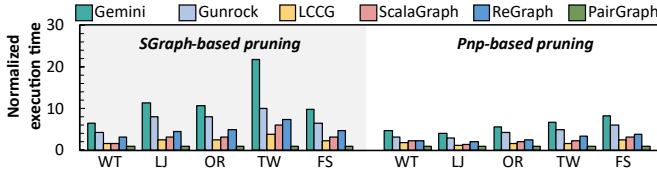


Fig. 9. Execution time normalized to PairGraph

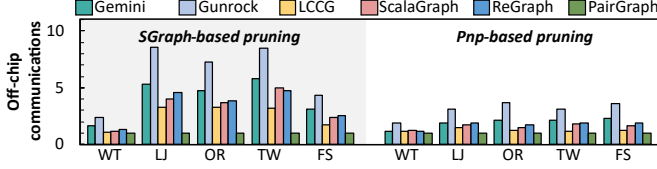


Fig. 10. Off-chip communications normalized to PairGraph

the data is loaded (①), the *Data Router* routes the data using a 4-bit routing code (see Fig. 8) encoded by the phase/state and priority signals. The phase/state signal determines the execution phase (RSG/OP) and data prefetch states, while the priority signal decides whether the data is in the shared regions. Based on the decoded information, the loaded data is sent to the *Index Buffer* (INB) (p_0), SCB (p_1), PCB (p_2), or directly bypassed (p_3) (②). For the case p_0 , if the region weight of the loaded vertex exceeds the threshold (i.e., the average region weight), the fetched vertex id is routed to the SCB via the DMux based on the priority signal (③), and the corresponding bit of the routing code in the *Encoder*, which represents priority, is set to 1 (④). Otherwise, the bit is set to 0, and the vertex id is bypassed.

D. SPM-customized Workloads Processing

The workloads in SPM consist of two parts: the original pairwise query workloads, and the propagating workloads for generating shared regions. The two types of workloads occur simultaneously at the propagating stage of the RSG phase. To process them, the edge should be processed by a function to determine whether the neighboring vertices should be activated. Additionally, the region weight needs to be propagated to the neighboring vertices for generating shared regions. To efficiently handle these workloads, we design *Scatter-Prune-Reduce Pipeline* (SPR) and *Propagating Lane* (PL) as shown in Fig. 6. The key insight is that there is no data dependency when processing the two types of workloads, which indicates that they can be processed in parallel.

As shown in Fig. 5(f), when a workload is dispatched to a PE, it is cut and sent to the *Scatter Unit* and *Propagator* simultaneously, enabling parallel processing by the SPR and PL. In the PL, the region weight is directly propagated to the neighboring vertex by the propagator (①). In the SPR, temporary data is firstly computed by the *Scatter Unit* using the property data of the source vertex and the edge weight. Then it is sent to the *Prune Unit*, where it is compared with the property data of neighboring vertices and existing computation results to determine whether to be pruned (②). If not pruned, the data is written to the vertex property array for the neighboring vertex by the *Reducer* (③), and the neighboring vertex is activated in the next iteration. Note that in the propagating stage of the RSG phase, the temporary data is compared with the tag instead of being sent to the *Prune Unit* to decide whether it should be reduced (④).

E. Correlation-aware Query Mapping

In the OP phase, queries are allocated to different CTiles for concurrent execution. However, directly mapping queries to CTiles according to the query count leads to load imbalance. As illustrated in Fig. 7(a), queries with various workloads are distributed across different CTiles based on the query count, often leading to load

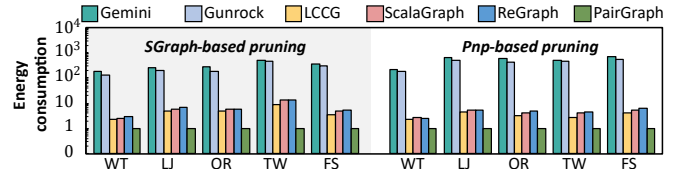


Fig. 11. Energy consumption normalized to PairGraph

imbalance due to the significant variation in workloads across queries. It means that, even when an equal number of queries are allocated to each CTile, some of them still become idle prematurely, which leads to low CTile utilization.

Fortunately, we discover the workloads of a pairwise query are strongly correlated with the minimum hop count between the source and destination vertices, as illustrated in Fig. 7(b). The reason is that a smaller hop count facilitates faster path discovery for pruning in pairwise query algorithms, thereby reducing overall number of convergent iterations. Based on this correlation, queries with the same total hop count are allocated to each CTile so as to achieve workload balance among CTiles. It should be noted that the minimum hop count can be easily recorded by the *propagator* in the RSG phase.

V. EVALUATION

A. Experimental Setup

PairGraph Simulation and Configurations. A cycle-accurate simulator is developed to model the behaviors of PairGraph. We wrote Verilog to implement the modules of PairGraph, and synthesized the RTL using Synopsys Design Compiler with TSMC 28nm standard library at 1 GHz to obtain the consumption of area and power for processing units. For the on-chip cache and buffers, we used CACTI [18] to model the area and power consumption. The behaviors of the off-chip memory access are modeled using Ramulator [17]. PairGraph consists of 4 CTiles, each containing 6 PEs. For on-chip memory on a CTile, we used 7 MB SCB and 1 MB PCB for data caching, 2 KB *Query Queue* for query scheduling, 4 KB *Fetched Buffer* for workload fetching, and 8 KB *Index Buffer* for region weight indexing.

Baselines. We compared PairGraph with a CPU-based system Gemini [2]; a GPU-based system Gunrock [20]; and three advanced graph processing accelerators: LCCG [5], ScalaGraph [12], and ReGraph [13]. We integrated two cutting-edge pruning functions: Pnp-based pruning function [3] and SGraph-based pruning function [4] into these baselines to implement pairwise query applications and adapted the baselines for concurrent query support. The CPU baseline is implemented on a server with two 32-core, 2.3 GHz Intel Xeon Gold 6314U processors, while the GPU baseline runs on an NVIDIA Tesla V100 GPU. Since the original configurations of the compared accelerators are different, we make the following efforts to ensure fair comparisons. First, we use the same on-chip cache capabilities (32 MB) as PairGraph for data caching. Second, we use the same number of compute units (1 GHz 24×SMT) to achieve equivalent computational performance. Last, we use main memory with the same performance (512 GB/s HBM 1.0) to ensure I/O fairness.

Datasets and Algorithms. We use six pairwise query algorithms, namely *Shortest Path* (PPSP) [4], *Widest Path* (PPWP) [4], *Narrowest Path* (PPNP) [4], *Breadth First Search* (BFS) [4], *Connectivity* (CN) [4], and *Reachability* (RC) [4], and five real-world graphs listed in Table I to evaluate the performance. To generate CPQ, the algorithms are submitted in a batch of 36 queries with each algorithm submitted six times, running between different pairs of source and destination vertices.

TABLE I
GRAPH DATASET USED IN THE EVALUATION

Datasets	Vertices	Edges	Directed
Wikipedia Talk (WT) [1]	2,394,385	5,021,410	Yes
LiverJournal (LJ) [1]	4,847,571	68,993,773	Yes
Orkut (OR) [1]	3,072,441	117,185,083	No
Twitter (TW) [1]	41,652,230	1,468,364,884	Yes
Friendster (FS) [1]	65,608,366	1,806,067,135	No

TABLE II
VARIOUS BATCHES OF QUERIES

Batch	The Batch of Query Algorithms
B_1	$\{36 \times \text{PPSP}\}$
B_2	$\{18 \times \text{PPSP}, 6 \times \text{PPWP}, 6 \times \text{PPNP}, 6 \times \text{BFS}\}$
B_3	$\{12 \times \text{PPSP}, 6 \times \text{PPWP}, 6 \times \text{PPNP}, 6 \times \text{BFS}, 6 \times \text{RC}\}$
B_4	$\{6 \times \text{PPSP}, 6 \times \text{PPWP}, 6 \times \text{PPNP}, 6 \times \text{BFS}, 6 \times \text{CN}, 6 \times \text{RC}\}$
B_5	$\{24 \times \text{PPSP}, 12 \times \text{BFS}\}$
B_6	$\{36 \times \text{BFS}\}$

B. Performance of PairGraph

Overall Performance. Fig. 9 shows that PairGraph outperforms the other solutions on all the datasets. Compared with Gemini, Gunrock, LCCG, ScalaGraph, and ReGraph, PairGraph improves the performance by $5.59 \times \sim 14.25 \times$, $3.76 \times \sim 7.58 \times$, $1.67 \times \sim 2.72 \times$, $1.93 \times \sim 4.26 \times$, and $2.66 \times \sim 4.28 \times$, respectively. These improvements are primarily due to the reduction in off-chip communications.

Off-chip Communications. As shown in Fig. 10, PairGraph reduces off-chip communications to 32.88%, 21.46%, 52.83%, 41.25%, and 38.76% of Gemini, Gunrock, LCCG, ScalaGraph, and ReGraph on average, respectively. The reduction stems from two key factors: First, PairGraph generates shared regions to effectively identify the graph structure data worth sharing, which can help to reduce fragmented data sharing during execution. Second, the data in the shared regions is fully shared by pinning in the SCB at each step of partition exploration, significantly enhancing data reuse.

Energy Savings. We use CPU Energy Meter [13] and nvidia-smi [13] to measure the energy consumption of Gemini and Gunrock, respectively. Fig. 11 shows that PairGraph reduces the energy consumption of Gemini, Gunrock, LCCG, ScalaGraph, and ReGraph by $443.62 \times$, $343.13 \times$, $4.15 \times$, $5.38 \times$, and $5.71 \times$ on average, respectively.

Power and Area Cost. The total power and area cost of PairGraph are 3.21 W and 11.23 mm^2 , respectively. The *Cache Module* uses most on-chip memory resources, occupying 71.2% of the area and consuming 26.2% of the power. The *Processing Module* occupies 13.4% of the area and consumes 52.1% of the power for workloads processing. The *Loading Module* takes up 11.1% of the area and uses 16.9% of the power for graph data prefetching and workloads dispatching. Lastly, the *Controlling Module* occupies 4.1% of the area and costs 4.8% of the power.

Sensitivity Analysis. Fig. 12(a) shows the performance of PairGraph under different thresholds for selecting shared regions. We evaluated the impact of setting the threshold to the minimum, maximum and average region weight. The results indicate that the scheme using the average region weight outperforms the others. Setting the threshold too high causes the *Data Router* to classify the majority of data as not requiring to be shared, while setting it too low hinders PairGraph’s ability to reduce fragmented sharing. Fig. 12(b) illustrates the sensitivity to the number of concurrent queries. The result shows that PairGraph achieves better performance as the number of queries increases. Fig. 12(c) shows the performance of PairGraph when processing various batches of pairwise query algorithms. We can

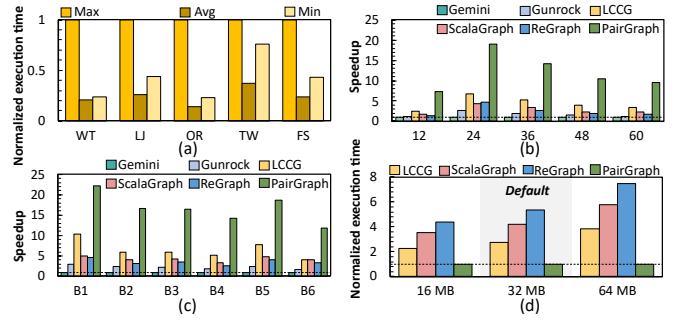


Fig. 12. Sensitivity studies: (a) Sensitivity to the threshold; (b) Sensitivity to the concurrent number on TW; (c) Speedups when using different batches of queries on TW; (d) Sensitivity to the cache size on TW

observe that PairGraph consistently outperforms other solutions for different cases listed in Table II. Fig. 12(d) shows that PairGraph outperforms other solutions due to higher cache utilization. And it obtains higher performance when the cache size is larger since more data can be effectively cached and shared by the queries.

VI. RELATED WORK

Pairwise Query. Several systems have been proposed to effectively handle a single pairwise query. *Hub*² [11] introduces a hub-centric strategy to limit the search scope of high-degree vertices, which significantly speeds up the PPSP process. Pnp [3] proposes a direction-aware pruning approach for processing pairwise queries with no preprocessing overhead. SGraph [4] applies both upper and lower bounds to impose stringent constraints so as to enhance pruning. However, these works still suffer from significant irregular memory accesses when processing CPQ.

Concurrent Graph Processing. Many concurrent graph processing schemes are proposed to address large-scale graph data. For distributed systems, Seraph [9] supports concurrent graph processing by decoupling the graph structure data. MultiLyra [21] and BEAD [23] improve batched query evaluation by distributing communication costs through graph sharing. For out-of-core systems on a single machine, prior works like CGraph [6] and GraphM [19] follow the idea of block-driven execution to exploit data sharing. Glin [8] introduces an inter-iteration alignment approach to enhance sharing efficiency. For in-memory graph processing, Congra [7] improves throughput by utilizing dynamic query scheduling and being mindful of atomic operations, while Krill [10] adopts an SAP model to simplify attribute data management and reduce memory accesses. Targeting hardware accelerators, LCCG [5] is proposed to eliminate redundant memory accesses along the graph topology so as to reduce off-chip communications. Although these approaches support the concurrent execution of typical point-to-all algorithms, they overlook the negative impact of the pairwise query’s unique search space on data sharing.

VII. CONCLUSION

This paper introduces PairGraph, a search-space-aware hardware accelerator for efficient CPQ processing. According to the characteristics of pairwise queries’ search spaces, it identifies the graph structure data traversed by most of the queries and coalesces the data in the shared cache to drive the CPQ processing, which fully exploits the temporal and spatial locality of traversal overlaps. As a result, PairGraph gains speedups of $5.59 \times \sim 14.25 \times$ and $3.76 \times \sim 7.58 \times$ compared with the state-of-the-art CPU-based system and GPU-based system, respectively. Compared with three cutting-edge accelerators, it gains speedups of $1.67 \times \sim 2.72 \times$, $1.93 \times \sim 4.26 \times$, and $2.66 \times \sim 4.28 \times$, respectively.

REFERENCES

- [1] J. Leskovec and A. Krevl, "SNAP Datasets: Stanford large network dataset collection," <http://snap.stanford.edu/data>, Jun.2014.
- [2] X. Zhu, W. Chen, W. Zheng, and X. Ma, "Gemini: A computation-centric distributed graph processing system," in *Proceedings of OSDI*, 2016, pp. 301–316.
- [3] C. Xu, K. Vora, and R. Gupta, "Pnp: Pruning and prediction for point-to-point iterative graph analytics," in *Proceedings of ASPLOS*, 2019, pp. 587–600.
- [4] H. Chen, M. Zhang, K. Yang, K. Chen, A. Zomaya, Y. Wu, and X. Qian, "Achieving sub-second pairwise query over evolving Graphs," in *Proceedings of ASPLOS*, 2023, pp. 1–15.
- [5] J. Zhao, Y. Zhang, X. Liao, L. He, B. He, H. Jin, and H. Liu, "LCCG: A locality-centric hardware accelerator for high throughput of concurrent graph processing," in *Proceedings of SC*, 2021, pp. 1–14.
- [6] Y. Zhang, X. Liao, H. Jin, L. Gu, L. He, B. He, and H. Liu, "CGraph: A correlations-aware approach for efficient concurrent iterative graph processing," in *Proceedings of ATC*, 2018, pp. 441–452.
- [7] P. Pan and C. Li, "Congra: Towards efficient processing of concurrent graph queries on shared memory machines," in *Proceedings of ICCD*, 2017, pp. 217–224.
- [8] X. Yin, Z. Zhao, and R. Gupta, "Glign: Taming misaligned graph traversals in concurrent graph processing," in *Proceedings of ASPLOS*, 2022, pp. 78–92.
- [9] J. Xue, Z. Yang, Z. Qu, S. Hou, and Y. Dai, "Seraph: An efficient, low-cost system for concurrent graph processing," in *Proceedings of HPDC*, 2014, pp. 227–238.
- [10] H. Chen, M. Shen, N. Xiao, and Y. Lu, "Krill: a compiler and runtime system for concurrent graph processing," in *Proceedings of SC*, 2021, pp. 1–16.
- [11] R. Jin, N. Ruan, B. You, and H. Wang, "Hub-accelerator: Fast and exact shortest path computation in large social networks," in *arXiv preprint arXiv:1305.0507*, 2013.
- [12] P. Yao, L. Zheng, Y. Huang, Q. Wang, C. Gui, Z. Zeng, X. Liao, H. Jin, and J. Xue, "ScalaGraph: A scalable accelerator for massively parallel graph processing," in *Proceedings of HPCA*, 2022, pp. 199–212.
- [13] X. Chen, Y. Chen, F. Cheng, H. Tan, B. He, and W. Wong, "ReGraph: Scaling graph processing on HBM-enabled FPGAs with heterogeneous pipelines," in *Proceedings of MICRO*, 2022, pp. 1342–1358.
- [14] A. Mukkara, N. Beckmann, M. Abeydeera, X. Ma, and D. Sanchez, "Exploiting locality in graph analytics through hardware-accelerated traversal scheduling," in *Proceedings of MICRO*, 2018, pp. 1–4.
- [15] N. Bell and M. Garland, "Implementing sparse matrix-vector multiplication on throughput-oriented processors," in *Proceedings of SC*, 2009, pp. 1–11.
- [16] J. Shun and G. E. Belloch, "Ligra: a lightweight graph processing framework for shared memory," in *Proceedings of PPOPP*, 2013, pp. 135–146.
- [17] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible DRAM simulator," *IEEE Computer Architecture Letters*, vol. 15, no. 1, pp. 45–49, 2016.
- [18] R. Balasubramanian, A. B. Kahng, N. Muralimanobhar, A. Shafiee, and V. Srinivas, "Cacti 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Transactions on Architecture and Code Optimization*, vol. 14, no. 2, pp. 1–25, 2017.
- [19] J. Zhao, Y. Zhang, X. Liao, L. He, B. He, H. Jin, H. Liu, and Y. Chen, "GraphM: an efficient storage system for high throughput of concurrent graph processing," in *Proceedings of SC*, 2019, pp. 1–14.
- [20] Y. Wang, A. Davidson, Y. Pan, Y. Wu, A. Riffel, and J. D. Owens, "Gunrock: A high-performance graph processing library on the GPU," in *Proceedings of PPOPP*, 2016, pp. 1–12.
- [21] A. Mazloumi, X. Jiang, and R. Gupta, "Multilyra: Scalable distributed evaluation of batches of iterative graph queries," in *Proceedings of BigData*, 2019, pp. 349–358.
- [22] Y. Huang, L. Zheng, H. Liu, Z. Zhou, D. Chen, P. Yao, Q. Wang, X. Liao, and H. Jin, "Meg2: In-memory Acceleration for genome graph analysis," in *Proceedings of DAC*, 2023, pp. 1–6.
- [23] A. Mazloumi, C. Xu, Z. Zhao, and R. Gupta, "BEAD: Batched evaluation of iterative graph queries with evolving analytics demands," in *Proceedings of BigData*, 2020, pp. 461–468.
- [24] H. Wang, G. Li, H. Hu, S. Chen, B. Shen, H. Wu, W. Li, and K. Tan, "R3: A real-time route recommendation system," in *Proceedings of the VLDB Endowment*, 2014, pp. 1549–1552.
- [25] K. Joseph and H. Jiang, "Content based news recommendation via shortest entity distance over knowledge graphs," in *Proceedings of WWW*, 2019, pp. 690–699.